



Note: In this article, we will confine ourselves to *local register allocation*.

Before we get into the details, let's get familiar with what register allocation means. Consider the sequence of instructions below as sample code that may be generated by the code-generation module.

```
1. MOVE 10, [a] ;
2. MOVE 11, [b] ;
3. ADD 12, 11, 10 ;
4. ADD 13, 10, 10 ;
5. ADD 14, 12, 10 ;
```



Note:

- The above instruction set contains virtual registers (10, 11, 12, 13, 14) as generated by the code-generation phase (register allocation has not yet been performed).
- [a]* and *[b]* imply memory locations. At the time of code generation, it is not known which address registers will be allocated, so symbolic names like *a* and *b* are used.
- Assume that the first operand is the destination operand, and the remaining are source operands.

Assuming that the number of physical registers available is three—*R0*, *R1* and *R2*, after register allocation the instructions will look like those in Figure 3.

Register allocation internals—the algorithm

Figure 4 is a block diagram overview of the algorithm used in the register allocation module.

The steps in the algorithm are:

Step 1: Compute live ranges of the virtual registers defined in the current basic block.

Step 2: Construct the Interference Graph.

Step 3: Perform graph colouring for the Interference Graph, to allocate physical registers to the virtual registers.

Step 4: If graph colouring fails, perform Register Spilling.

We examine each of these steps below.

Step 1: Perform live range analysis

The live range of a virtual register: If a virtual register is defined at instruction *u*, and it is last used at instruction *v*, then its live range will be *{u, v-1}*.

To understand this, consider the example in Figure 5.

Instruction 1 defines virtual register *10*, which was last used in Instruction 5. Hence, the live range of virtual register *10*

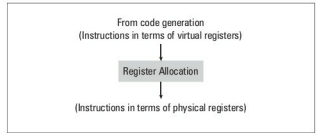


Figure 1: Register allocation

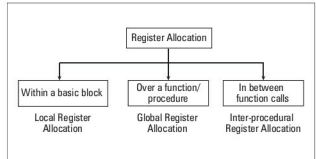


Figure 2: Types of Register Allocation

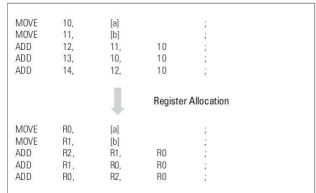


Figure 3: Instructions after Register Allocation

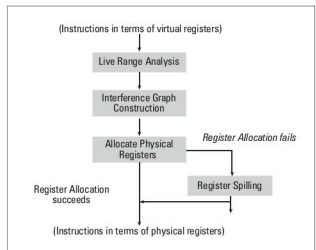


Figure 4: Block diagram of Register Allocation